

Mock Paper 2B — Algorithms & Programming (H446/02)

Time allowed: 2 hours 30 minutes · Total: 140 marks · Answer all questions.

Instructions

- Use **OCR Exam Reference Language** or a **high-level language** for all code.
- **Show all traces** in full — give the complete variable / stack / table state at every step, not just the final answer.
- Quality of written communication is assessed in the extended-response question.
- Mark your work against the **mark scheme** at the end of this paper.
- Each part is tagged with its mark total [n] and the dominant assessment objective (A01 / A02 / A03) .

Questions

Question 1 — Computational thinking applied to a scenario [11]

A logistics company runs **GridStore**, an automated warehouse. Hundreds of robots collect items from shelves and bring them to packing stations. A central controller plans each robot's route, avoids collisions, and reorders jobs when an urgent order arrives.

(a) The design team models the warehouse floor as a grid of square cells, each marked as *free* or *blocked*, and ignores the physical height of shelving and the colour of the robots. Name the computational thinking technique being used and explain, with reference to GridStore, **why** it is helpful here. [3] (AO2)

.....

.....

.....

.....

(b) The job of "fulfil one customer order" is broken down into smaller tasks. Identify **two** sensible sub-problems of fulfilling an order and, for each, state one input and one output. [4] (AO2)

.....

.....

.....

.....

.....

(c) Two robots simultaneously plan routes that pass through the same cell at the same time-step. Explain why this is an example of a problem requiring **concurrency** to be managed, and describe **one** consequence if it is not managed. [2] (AO2)

(d) The controller must guarantee a fair, predictable response time when reordering jobs. State **one** reason why **abstraction by generalisation (categorisation)** of jobs into "standard" and "urgent" types helps the controller meet this requirement. [2] (AO2)

Running total: 11

Question 2 — Programming techniques [12]

A developer is writing utility subroutines for GridStore.

(a) State **one** difference between an **iterative** solution and a **recursive** solution to the same problem. [1] (AO1)

(b) The subroutine below is written in OCR Exam Reference Language. Global variable `count` is declared outside the subroutine.

```
count = 0

procedure adjust(byRef arr, byVal step)
    for i = 0 to arr.length - 1
        arr[i] = arr[i] + step
        count = count + 1
    next i
endprocedure

nums = [10, 20, 30]
adjust(nums, 5)
print(nums)
print(count)
```

State exactly what is printed, in order, and explain your answer with reference to **pass by reference**, **pass by value** and **variable scope**. [5] (AO2)

(c) Explain what is meant by the **scope** of a variable and give **one** disadvantage of relying on a **global** variable such as `count` above. [3] (AO2)

(d) Rewrite the loop below so that it has identical behaviour but uses a **condition-controlled (while)** loop instead of a count-controlled loop: [3] (AO2)

```
total = 0
for k = 1 to 5
    total = total + k
next k
```

Running total: 23

Question 3 — Recursion and the call stack [10]

The recursive function `power` is defined below.

```
function power(base, exp)
  if exp == 0 then
    return 1
  else
    return base * power(base, exp - 1)
  endif
endfunction
```

(a) Identify the **base case** and the **recursive (general) case** of this function. [2] (AO1)

(b) The call `power(3, 4)` is made. Using a **call-stack** trace, show each frame pushed onto the stack and the value returned as each frame is popped (unwinds). [5] (AO2)

(c) State the **final value** returned by `power(3, 4)`. [1] (AO2)

(d) A colleague suggests removing the `if exp == 0` base case so that the function "just keeps multiplying". Explain what happens at run-time if the base case is removed, and name the resulting error. [2] (AO2)

Running total: 33

Question 4 — Object-oriented programming [14]

GridStore models its robots with classes. Write/complete the definitions below (OCR Exam Reference Language or a high-level language).

(a) Define a class `Robot` with **private** attributes `id`, `batteryLevel` (0–100) and `carrying` (boolean). Write a **constructor** that sets `id` from a parameter, sets `batteryLevel` to 100 and sets `carrying` to `false`. [5] (AO3)

(b) Add two methods to `Robot`: - `pickUp()` — if the robot is **not** already carrying, set `carrying` to `true` and return `true`; otherwise return `false`. - `drain(amount)` — reduce `batteryLevel` by `amount`, but never below 0. [5] (AO3)

(c) A class `ChargingRobot` **inherits** from `Robot` and adds a method `recharge()` that sets `batteryLevel` back to 100. Write `ChargingRobot` so that it inherits from `Robot` and adds this method. [2] (AO3)

(d) State the name of the OOP relationship between `ChargingRobot` and `Robot`, and give **one** advantage to the developers of using inheritance here. [2] (AO2)

Running total: 47

Question 5 — Computational methods [11]

(a) GridStore's route planner must plan around shelves, robots and walls. Explain what is meant by a **problem being solvable but intractable**, and give **one** example of a problem that is computable in principle but **not** in reasonable time for large inputs. [3] (AO2)

(b) Explain the difference between an **algorithm that guarantees the optimal solution** and a **heuristic**, and give **one** reason a heuristic could be preferred for GridStore's real-time route planning. [3] (AO2)

(c) The route planner must satisfy several conditions at once: each cell visited must be free, the path must not exceed the robot's remaining battery, and no two robots may occupy a cell at the same time-step. Name the computational method that describes solving a problem subject to a set of such restrictions. [1] (AO1)

(d) Explain what is meant by a **divide-and-conquer** approach, and state **one** standard algorithm (other than a sorting algorithm) that uses it. [2] (AO2)

(e) State **one** benefit of using **caching** of previously planned routes between common start and end cells, and **one** situation in which the cached route could become invalid. [2] (AO2)

Running total: 58

Question 6 — Big O analysis [11]

(a) Explain what **Big O notation** describes, and state why it expresses the **worst case** as a function of input size n rather than an exact running time in seconds. [2] (AO2)

(b) State the **worst-case** time complexity, using Big O, of each of the following: - (i) linear search of an unsorted list of n items - (ii) binary search of a sorted list of n items - (iii) merge sort of n items - (iv) accessing element i of an array by its index [4] (AO1)

(c) Consider the algorithm below.

```
i = n
while i > 1
    print(i)
    i = i DIV 2
endwhile
```

State its time complexity in Big O and **justify** your answer. [3] (AO2)

(d) Algorithm X runs in $O(n^2)$ and algorithm Y runs in $O(2^n)$. State which scales worse for large n , and explain why. [2] (AO2)

Running total: 69

Question 7 — Searching algorithms [10]

(a) State **one** precondition that must hold before a **binary search** can be used. [1] (AO1)

(b) Trace a **binary search** for the value **16** in the sorted list below (indices 0–8). At each step show `low`, `high`, `mid` and `list[mid]`, and state the total number of comparisons of `list[mid]` made. Use `mid = (low + high) DIV 2`. [5] (AO2)

Index:	0	1	2	3	4	5	6	7	8
Value:	4	9	16	23	31	38	52	61	77

(c) Complete the **binary search** function below so that it returns the **index** of `target` if found, or `-1` if not found. [4] (AO3)

```
function binarySearch(list, target)
  low = 0
  high = list.length - 1
  while low <= high
    mid = (low + high) DIV 2
    // ... complete: three branches ...
  endwhile
  // ... complete ...
endfunction
```

Running total: 79

Question 8 — Sorting algorithms [13]

(a) A list contains [6, 3, 8, 2, 5]. Perform a **bubble sort**, writing out the **full state of the list after each complete pass**. State the pass on which your implementation could first detect that the list is sorted (no swaps). [5] (AO2)

(b) State the **worst-case** and **best-case** time complexity of bubble sort (assuming a swap-detection optimisation), and describe the input that produces the **best** case. [3] (AO2)

(c) Quick sort sorts the list [7, 2, 9, 4, 3, 8, 1] using the **last element of each (sub)list as the pivot**. Show the partitioning at each level: for each partition state the pivot, the resulting arrangement, and the left/right sublists produced, until the list is fully sorted. [3] (AO3)

(d) Quick sort has an average-case complexity of $O(n \log n)$ but a worst case of $O(n^2)$. Describe the kind of input/pivot choice that causes quick sort's **worst case**, and state **one** advantage quick sort has over merge sort. [2] (AO2)

Running total: 92

Question 9 — Data structure algorithms [12]

(a) A **circular queue** is implemented in an array `q` of size **5** (indices 0–4) with pointers `front` and `rear` and a `size` counter. Complete the `enqueue` procedure and the `dequeue` function below, including handling of the **full** and **empty** conditions. [5] (AO3)

```
procedure enqueue(value)
    if size == 5 then
        print("Queue full")
    else
        // ... complete: advance rear with wrap-around, store value, update size ...
    endif
endprocedure

function dequeue()
    if size == 0 then
        return "Queue empty"
    else
        // ... complete: read from front, advance front with wrap-around, update size, return
    endif
endfunction
```

(b) Starting from an **empty** circular queue of size 5 (`front = 0` , `rear = -1` , `size = 0`), the following operations are performed in order:

```
enqueue(A), enqueue(B), enqueue(C), dequeue(), enqueue(D), dequeue(), enqueue(E)
```

State, **after all operations**, the value of `front` , the value of `rear` , and the values currently stored in the queue in order from front to rear. [3] (AO2)

(c) A **singly linked list** stores the nodes below. `head` points to the node at index 2. Each node holds (data, pointer) , where the pointer is the index of the next node and `pointer = null` marks the end.

Index:	1	2	3	4
Data:	"Oslo"	"Lima"	"Cairo"	"Quito"
Pointer:	4	3	1	null

(i) State the order in which the data is read when the list is **traversed** from `head` . [2] (AO2)

(ii) The node "Cairo" (index 3) is to be **deleted** from the list. State which single pointer changes and its new value, and give the new traversal order. [2] (AO3)

Running total: 107

Question 10 — Trees and traversals [11]

The values 55, 40, 65, 25, 48, 60, 80, 45 are inserted, in that order, into an initially empty **binary search tree (BST)**.

(a) Draw the resulting BST. [3] (AO3)

(b) Give the **pre-order**, **in-order** and **post-order** traversal output of your tree. [3] (AO2)

(c) A program needs to output the stored values in **descending** order. Describe how a traversal of the BST could be modified to achieve this directly, without sorting the output. [2] (AO2)

(d) State the **best-case** and **worst-case** time complexity of searching for a value in a BST, and describe the **shape** of tree that gives each case. [3] (AO2)

Running total: 118

Question 11 — Shortest path (Dijkstra / A*) [12]

Consider the weighted, **undirected** graph below.

$$A-B = 4, \quad A-C = 3, \quad B-C = 1, \quad B-D = 2, \quad C-D = 5,$$
$$C-E = 8, \quad D-E = 2, \quad D-F = 6, \quad E-F = 3$$

This image shows a full page of primary-ruled paper. It features ten sets of horizontal lines across the page. Each set consists of three lines: a solid top line, a dashed middle line, and a solid bottom line. The lines are evenly spaced and extend from the left margin to the right edge of the page. There are no margins or other markings present.

Question 12 — Design and evaluation (extended response) [13]

GridStore (from Question 1) must store the live status of **every robot** (location, battery level, whether it is carrying an item, whether it is free) and, for each new order, repeatedly find the **nearest free robot** with **sufficient battery** to reach the pickup shelf. Robots change status many times per second, and thousands of orders arrive per minute.

Discuss how you would **design** the data storage and **searching** strategy for this problem. Your answer should:

- identify **suitable data structure(s)** for storing robot status and justify the choice, considering that status updates are very frequent;
- describe a **searching / algorithmic approach** for finding the nearest suitable robot, referring to relevant computational methods and/or graph/search techniques (e.g. spatial indexing, Dijkstra/A*, heuristics);
- analyse the **time-complexity** implications of your approach as the number of robots grows;
- **evaluate** at least one **trade-off** in your design (e.g. speed vs memory, accuracy vs responsiveness, simplicity vs scalability) and reach a justified conclusion.

The quality of your written communication will be assessed. **[13] (AO3)**

[illegible]

Running total: 140

